

[Guide Menu](#)[On this page](#)

System design

Travel Booking (e.g. Airbnb) ★ Premium

🔥 Medium ⌚ 30 mins 🏆 1.03k done

[Report an issue](#)

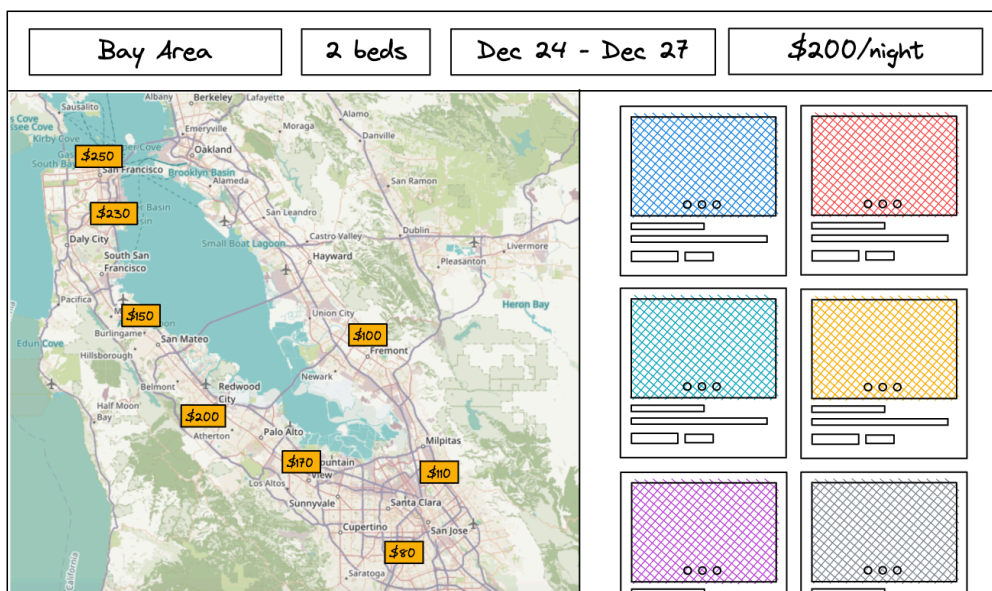
Asked at these companies

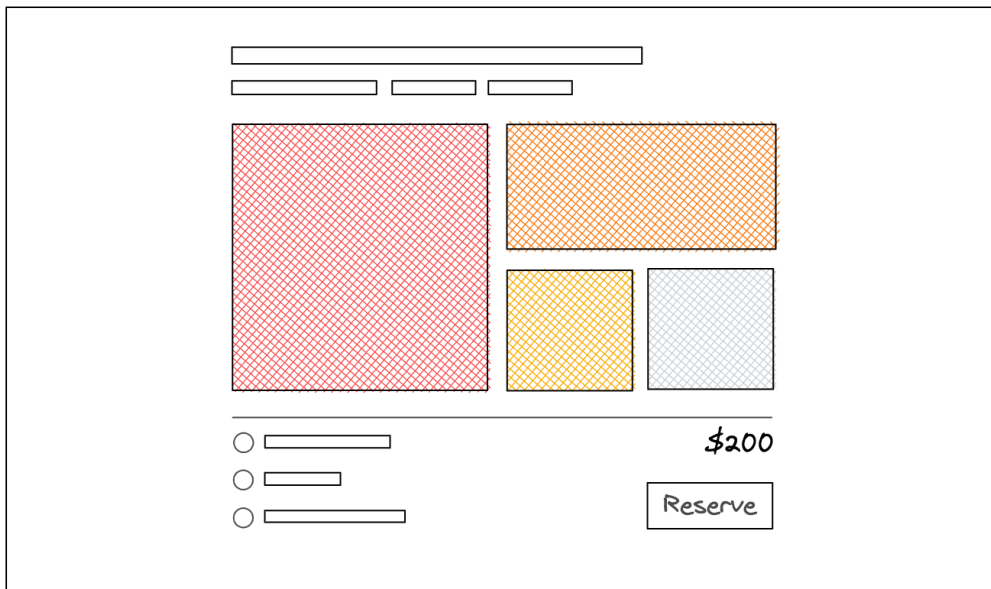
 Airbnb**SWAG**overflow Visit the ultimate swag store for Front End Engineers! →[Dashboard](#) [Prepare](#)

New questions added in Oct 2025



Design a travel booking website that allows users to search for accommodations and make a reservation.





Real-life examples

- [Airbnb](#)
- [Booking.com](#)
- [Expedia](#)
- [TripAdvisor](#)

Requirements exploration

We scope the problem by clarifying which user journeys are in play, which devices to target, and which global concerns, including SEO, localization, and currency, shape the design.

What are the core features to be supported?

- Search and browse accommodation listings.
- Viewing accommodation details such as price, location, photos, and amenities.
- Make reservations for accommodations.

What do the user demographics look like?

International users of a wide age range: US, Asia, Europe, etc.

What are the non-functional requirements?

Each key public route should meet the recommended p75 Core Web Vitals targets of $LCP \leq 2.5$ seconds, $INP \leq 200$ ms, and $CLS \leq 0.1$, measured separately for mobile and desktop in key markets. Search and reservation APIs should also have route-specific latency and error-rate targets.

What devices will the website be used on?

All possible devices: laptops, tablets, mobile, etc.

Do users have to be signed in?

Anyone can search for listings and browse details, but users need to be logged in to make a booking.

Summary

From the above, we can gather that the most important aspects of the website are:

- **Search engine optimization:** Travel sites should aim to rank high on search engines since organic search is one of the primary discovery mechanisms.
- **Performance:** Performance is known to affect conversions. For websites where the aim is to get customers to make purchases, performance is very important.
- **Internationalization:** The users are from many different countries and from various age groups. To capture a larger market, the website should be translated and localized.
- **Device support:** The site should work well on a variety of devices since the user demographics are very broad.
- **Pricing scope:** Assume the displayed price for the selected stay does not change during the reservation attempt.

The requirements and nature of a travel booking website are very similar to the E-commerce website system design, so there is a high degree of overlap between the

solutions. In this solution, we will cover the unique aspects of travel booking websites that aren't found on e-commerce websites.

★ **SEO and performance are the thesis, not nice-to-haves**

Travel sites depend on organic search for discovery and on conversion-sensitive load times to turn browsers into bookers. Every design choice below, including rendering strategy, URL shape, and image loading, should be evaluated against whether it helps public pages get indexed and whether it lets results render quickly on first load.

Exercise Beta

A traveler-facing booking design focuses on the journey from search through reservation. Which boundaries keep the work focused? Select all that apply.

- ☐ 1 Cover search and filtering, listing details and availability, reservation, and the resulting confirmation.
- ☐ 2 Require live bidding and repeated price negotiation while a traveler completes the reservation form.
- ☐ 3 Hold the displayed stay price fixed during one reservation attempt.
- ☐ 4 Treat SEO, internationalization, and route-specific performance and reliability as first-version constraints for public search and listing pages.
- ☐ 5 Allow anonymous public discovery across devices while requiring sign-in when the traveler books.
- ☐ 6 Include host listing management, payout reporting, and property onboarding in the first-version traveler experience.



Reveal answer

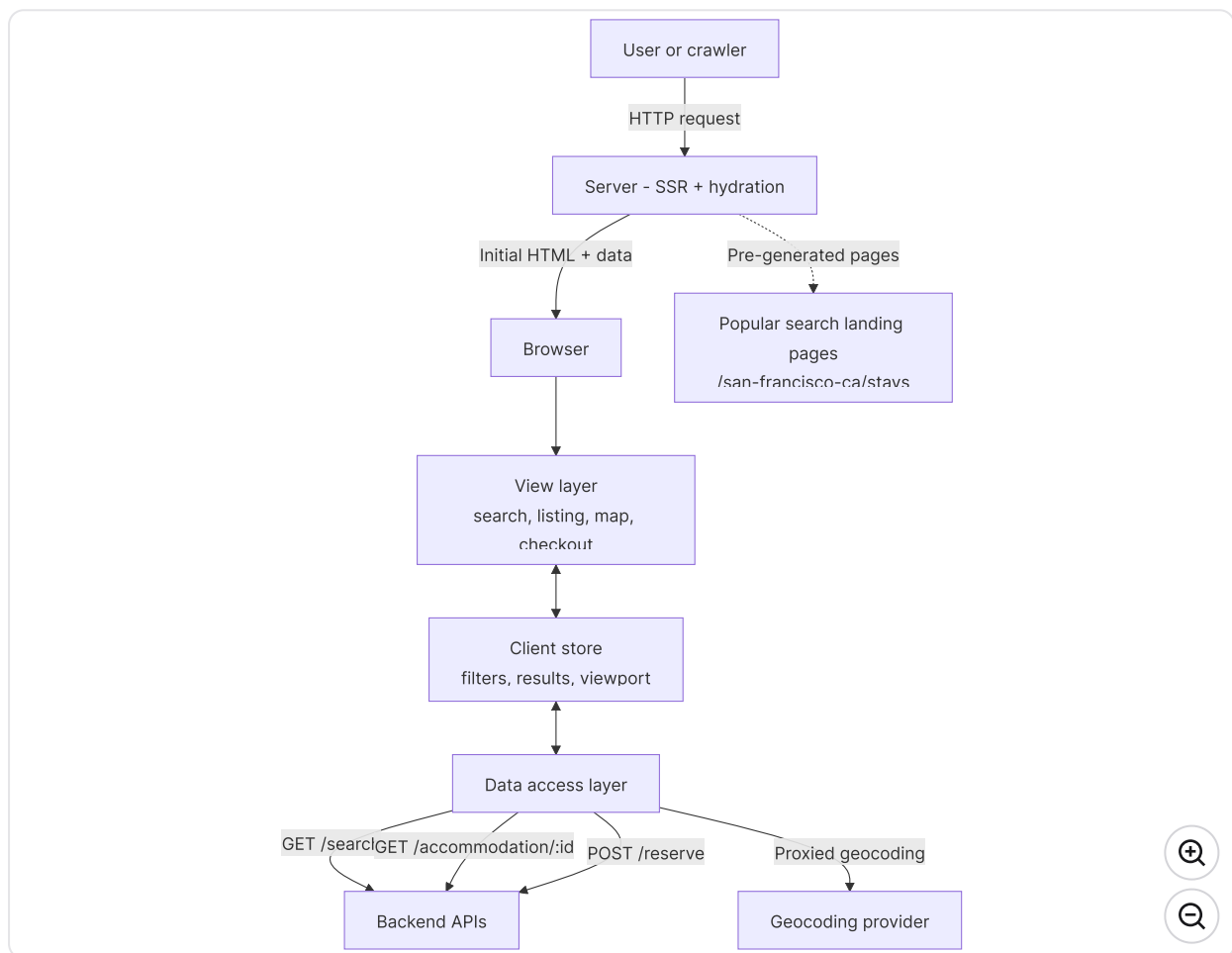
Check answer ←

Architecture / high-level design

The key architectural decisions for a travel booking site, including where to render, how to navigate, and how to structure the client, all flow from two realities: public pages must rank on search engines, and users compare many listings in parallel.

Server-side rendering (SSR) or Client-side rendering?

As with the E-commerce website system design, performance and SEO are critical. SSR or pre-rendering is a strong default for durable public destination and listing routes; private and highly interactive routes can choose a different rendering mode when its tradeoffs are acceptable.



Travel booking rendering and navigation topology

✓ Default to SSR with hydration for public listing and search pages

Pure client-side rendering is a weak default for durable public search and listing routes because they need to be crawlable by search engines and fast on the first paint. Render their initial HTML on the server and hydrate for interactivity rather than shipping an empty shell that fills in via JavaScript.

Single-page application (SPA) or Multi-page application (MPA)?

Travel users often compare several listings after narrowing the search results, which is why many travel sites open listing details in a new tab. However, the product should use ordinary listing links by default so users retain control over that choice. Preserve filters in the URL and restore the result cursor, map viewport, stable listing anchor, and scroll offset on back/forward navigation.

Hence, whether the site is an SPA or MPA matters less than choosing an appropriate rendering mode for each route. Both can be viable depending on how important the subsequent navigation experience is, and travel websites are one of the few products where an MPA architecture is still viable.

However, travel websites have a fair bit of interaction (e.g. changing search filters, interacting with the map, expanding accommodation details, etc.). Using UI frameworks is crucial for writing maintainable client-side code. React, Vue, and Angular are top choices for doing so.

For an SSR + interaction-heavy use case, we can leverage universal/isomorphic rendering (or SSR with hydration). In universal rendering, the server renders the full initial HTML, but after that, rendering and navigation become client-side. JavaScript event handlers are attached to the interactive elements within the HTML (hydration).

Airbnb is one of the pioneers of building universal/isomorphic apps with the invention of Rendr, a library that renders Backbone.js apps on the client and the server. Rendr is a historical example; modern React frameworks such as Next.js provide SSR with hydration.

Illustrative travel-site architectures

The following table is a simplified historical illustration of possible rendering choices, not a current stack audit:

	App Type	Rendering	UI Framework
Airbnb	SPA (some routes)	SSR	React

	App Type	Rendering	UI Framework
Booking.com	MPA	SSR	React
Expedia	MPA	SSR	React
TripAdvisor	SPA (some routes)	SSR	React

These examples show that SSR and React have been common choices, not that either is a universal requirement. Both SPA and MPA examples appear because rendering and navigation architecture are separate decisions.

For simplicity, and since SPAs are covered commonly in other questions, the discussion below assumes that we'll be using an MPA architecture with server-side rendering + hydration due to the need for interactivity after initial load.

References

- [Rearchitecting Airbnb's Frontend | Airbnb Engineering Blog](#)
- [Server Rendering, Code Splitting, and Lazy Loading with React Router v4 | Airbnb Engineering Blog](#)
- [Rendr: Run your Backbone apps in the browser and Node | Airbnb Engineering Blog](#)
- [Isomorphic JavaScript: The Future of Web Apps | Airbnb Engineering Blog](#)
- [Breaking the Monolith — Modular redesign of Agoda.com | Agoda Engineering & Design](#)

Exercise 1 of 2 Beta

Which rendering approach best supports indexable accommodation pages while preserving filter and map state during client navigation?

1

Server-render indexable destination and accommodation content, then hydrate search, filters, galleries, and booking interactions.

2

Client-render destination and listing bodies, but server-render their metadata and social preview tags.

3

Server-render public routes and replace the page for every filter, gallery, and booking interaction.

4

Statically generate every listing with its current price and calendar, refreshing pages only during scheduled deployments.



Reveal answer

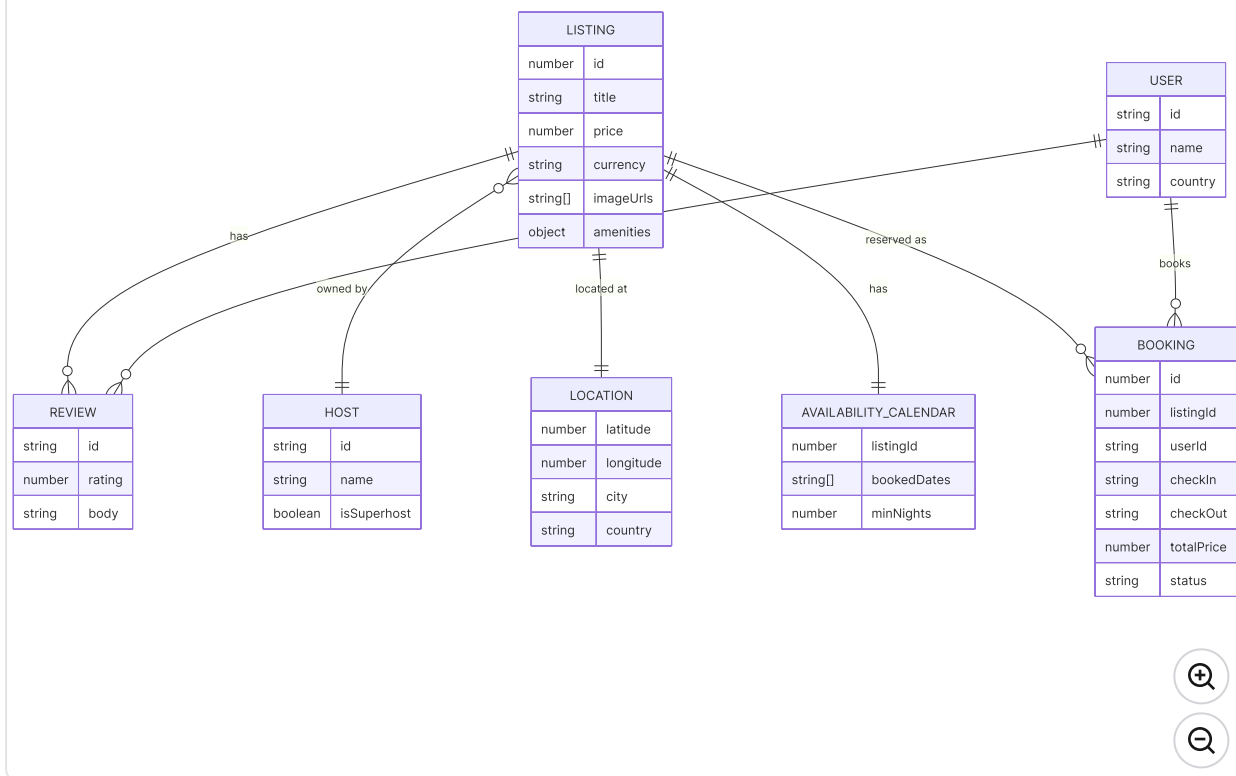
Check answer ↩

Data model

Entity	Source	Belongs to	Fields
Search Params	User input (Client)	Search/Listing Page	City/Geolocation/Radius Date Range, Number of People, Amenities, etc.
ListingResults	Server	Search/Listing Page	query key, ordered listing IDs, pagination metadata, request status
ListingItem	Server	Search/Listing Page, Details Page	stable <code>id</code> , <code>title</code> , <code>price</code> , <code>currency</code> , <code>image_urls</code> , <code>amenities</code> ; the detail response enriches the same entity

We have omitted the entities related to payments and addresses since they are covered in **E-commerce website system design**. If the interviewer wants you to focus on the checkout flow as well, do mention those entities.

Across search, map, detail, and checkout surfaces, the client store holds a compact set of related entities. Listings reference hosts and locations, and a booking ties together a user, a listing, and a date range.



Client-side entity relationships for listings and bookings

Exercise 1 of 2 Beta

Which state boundaries keep travel search and booking coherent? Select all that apply.

- 1 Represent destination, dates, party size, filters, and sort in shareable route state where appropriate. ☐
- 2 Store complete mutable listing snapshots inside each search page so cached pages can render independently. ☐
- 3 Store each search result set as a query key, ordered listing IDs, pagination metadata, and request status. ☐
- 4 Parse the localized price and date labels from the selected card when constructing the reservation request. ☐
- 5 Store accommodations and reusable entities by stable ID. ☐



Reveal answer

Check answer ↩

Interface definition (API)

We will need the following HTTP APIs:

1. **Search:** Fetch accommodation listings given some search/filter parameters. The results are rendered on the map and/or in a list view.
2. **Listing details:** Fetch the details for an accommodation listing.
3. **Reserve:** Make a reservation for an accommodation.

Search accommodations

Property	Value
HTTP Method	GET
Path	/search
Description	Returns a list of accommodations that match the search query.

Parameters

Parameter	Type	Description
Size	number	Number of results per page
Page	number	Page number to fetch
Guests	number	Number of staying guests

Parameter	Type	Description
Locale	string	Presentation preference for language and formatting
Currency	string	Optional ISO currency preference; the server still returns the reservation currency
Location	mixed (see below)	Location of the search
Date range	mixed (see below)	Date range to occupy the accommodation
Amenities	mixed (see below)	Amenities criteria

Location: The exact format of the `location` field depends on the business requirements, but in general, we need to tell the server the desired boundary so that results within the boundary will be shown. There are two common ways to represent boundaries: (1) center position + radius, and (2) boundary coordinates.

1. **Center position + radius:** The radius can be in miles/km and the center position can be either of:
 - **Free text search:** This is any string containing a location, e.g. "San Francisco", "New York City". An external location service will be needed to convert the string into geographic coordinates (via geocoding). Geocoding is preferably done on the server to prevent API abuse.
 - **Geolocation/Coordinates:** This format is useful for map-based UIs where users can pan/zoom the map to search for listings within the area, or when users allow the app to access their current location to search around them.

! Do not call the geocoding provider directly from the client

Shipping the geocoding API key to the browser exposes it to abuse and quota exhaustion, and makes it trivial for third parties to scrape the provider on your bill. Proxy geocoding through your own server so the key stays private and the service can be rate limited and cached per user. 2. **Boundary coordinates:** For map-based

UIs, we can use the coordinates of the 4 corners of the presented map as the boundary area.

If geographic search becomes a deeper discussion, normalize free text to a stable place ID and define one longitude/latitude boundary order so URL, cache, map, and list requests share the same identity.

It's likely that the search API will have to support both formats if the website has different search UIs:

- Landing pages for travel sites usually have a search bar with compulsory parameters like date range and number of guests. Such a UI will need the free text search location format.
- Results pages with maps will use the geolocation/boundary coordinates format.

Date range: There are several date range formats to choose from, and none are clear winners:

Format	Example	Pros	Cons
Array/tuple	<code>['2022-12-24', '2022-12-27']</code>	Short	Unclear. Has to be encoded
Object	<code>{ check_in: '2022-12-24', check_out: '2022-12-27' }</code>	Clear	Long. Has to be encoded
Separate query parameters	<code>check_in</code> and <code>check_out</code>	Don't need encoding	Two query params vs one

The array format is the least clear of the three, and the object requires encoding to be used in `GET`, so separate query parameters would be the most recommended.

Amenities: Structuring the query parameters for amenities has the same issues as the date range.

Format	Example	Pros	Cons
Object	<code>{ breakfast: true, washer: true }</code>	Clear	Long. Has to be encoded
Separate query parameters	<code>breakfast</code> and <code>washer</code>	More readable	N query params vs One

In this case, due to the sheer number of query parameters if each amenity criterion is a separate query parameter, the object format is better because:

- Putting each criterion in its own query parameter makes the query string really long, and the hierarchy is lost.
- There's a possibility of query parameter name collisions with the non-amenities fields. This can be solved by prefixing all the amenity fields with `amenities_` (e.g. `amenities_breakfast` and `amenities_washer`).
- Moreover, some amenities criteria have non-primitive values, which will require URL encoding and decoding anyway.

There's no fixed standard on how to encode arrays and objects into the query string of URLs, and the format tends to be implementation-dependent. The most important thing is to maintain a consistent format between the server and the client.

Sample response

```
1  {
2    // Pagination metadata.
3    "pagination": {
4      "size": 5,
5      "page": 2,
6      "total_pages": 15,
7      "total": 74,
8    },
9    "results": [
10     {
11       "id": 561602, // Accommodation ID.
12       "title": "Great view in the Mission, 15 mins by bus downtown",
13       "images": [
14         "https://www.greathotels.com/img/1.jpg",
```

```
15     "https://www.greathotels.com/img/2.jpg",
16     "https://www.greathotels.com/img/3.jpg",
17     "https://www.greathotels.com/img/4.jpg",
18   ],
19   "rating": 4.82,
20   "coordinates": {
21     "latitude": 37.74403,
22     "longitude": -122.41755,
23   },
24   "price": 200,
25   "currency": "USD",
26 },
27 // ... More accommodation results.
28 ],
29 }
```

Offset-based pagination supports numbered result pages. Use a deterministic sort with a unique tie-breaker, while accepting that results can still shift when ranking, price, or availability changes.

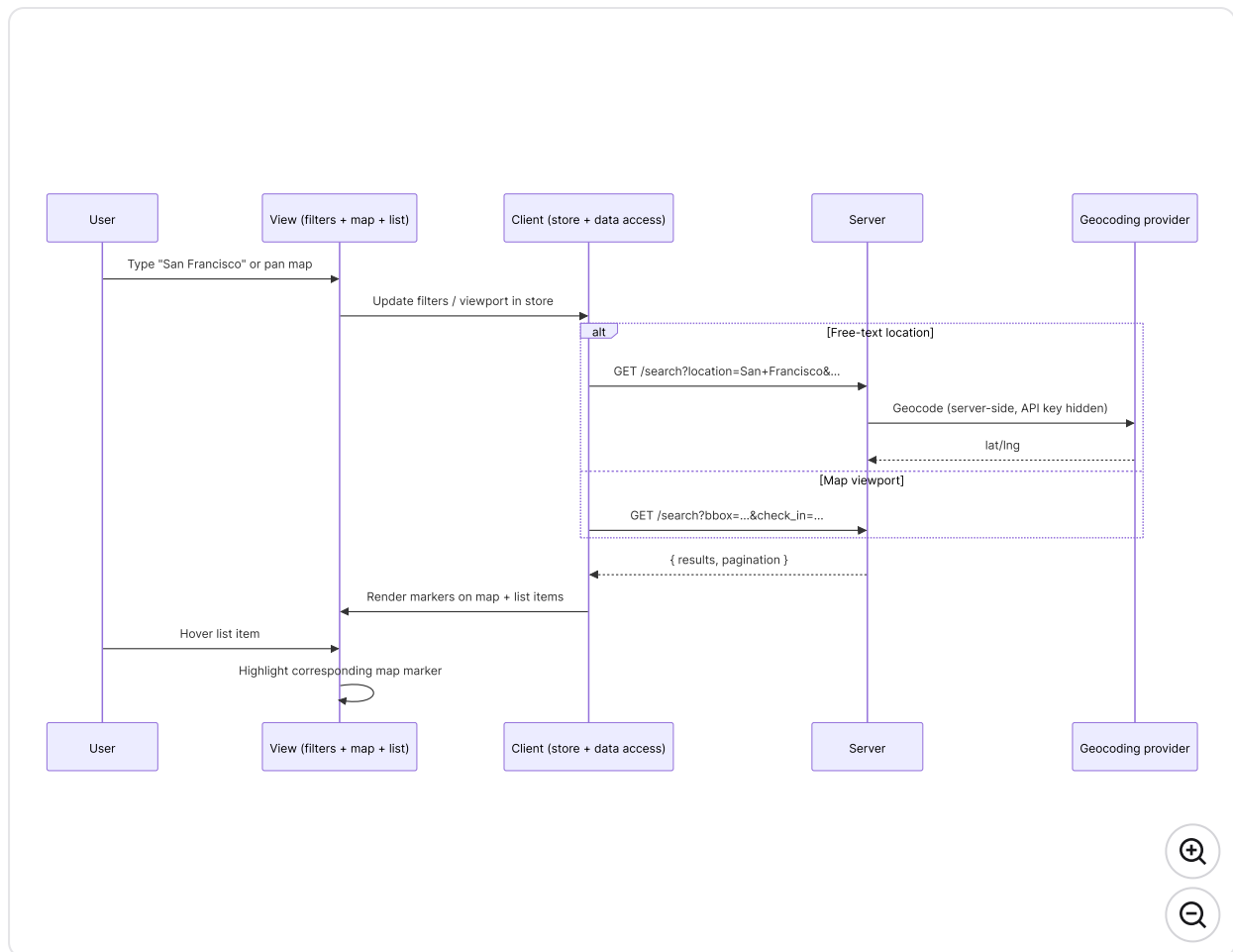
We use offset-based pagination here as opposed to cursor-based pagination because:

1. Having page numbers is useful for navigating between search results and jumping to specific pages.
2. Accommodation results do not suffer from the stale results issue that much because new listings are not added that quickly/frequently.
3. It's useful to know how many total results there are.

For a more in-depth comparison between offset-based pagination and cursor-based pagination, refer to the [News Feed system design article](#).

If infinite scroll is desired, then a cursor-based pagination approach might be required. Offset-based pagination can still be used, but the client will need to go through the trouble of filtering out duplicate results.

On a typical search-with-map surface, filter changes, map panning, and free-text input all converge on the same `/search` call, with geocoding proxied through the server and the map + list views re-rendered from the same result set.



Search with map and filters flow

Derive a normalized request key from committed URL state, debounce only transient text or viewport input, abort superseded reads where practical, and accept a response only when its generation/key is still current. Keep previous data visible while exposing loading, empty, partial, offline, and retry states.

✓ Default to offset pagination for travel search, cursor only if infinite scroll

Numbered pages can use offset pagination with a deterministic sort. If a continuous or frequently re-ranked result set needs stable continuation, use an opaque continuation token instead.

Fetch accommodation details

The details endpoint can serve SSR and client navigation through one documented response contract; it does not depend on forcing a new browsing context.

Property	Value
HTTP Method	GET
Path	/accommodation/{accommodationId}
Description	Fetches the details of an accommodation listing.

Parameters

Parameter	Type	Description
accommodationId	number	ID of accommodation to be fetched
check_in	string	Optional property-local check-in date used to check availability
check_out	string	Optional property-local check-out date used to check availability
guests	number	Optional guest count used to check availability
locale	string	Presentation preference for language and formatting
currency	string	Optional ISO currency preference

Sample response

```
1  {
2    "id": 561602, // Accommodation ID.
3    "title": "Great view of Brannan Street, 15 mins by bus downtown. Bed a
4    "images": [
5      "https://www.greathotels.com/img/1.jpg",
6      "https://www.greathotels.com/img/2.jpg",
7      "https://www.greathotels.com/img/3.jpg",
```

```
8     "https://www.greathotels.com/img/4.jpg",
9   ],
10  "rating": 4.82,
11  "coordinates": {
12    "latitude": 37.74403,
13    "longitude": -122.41755,
14  },
15  "price": 200,
16  "currency": "USD",
17  "time_zone": "America/Los_Angeles",
18  "amenities": {
19    "breakfast_provided": true,
20    "internet": true,
21    "washer": true,
22    "dryer": false,
23    // Any additional amenities details.
24  },
25  "house_rules": "...",
26  "contact_email": "...",
27  // Any additional details.
28 }
```

Make reservation

Property	Value
HTTP Method	POST
Path	/reserve
Description	Reserve an accommodation.

Parameters

Parameter	Type	Description
accommodation_id	number	ID of accommodation to be reserved

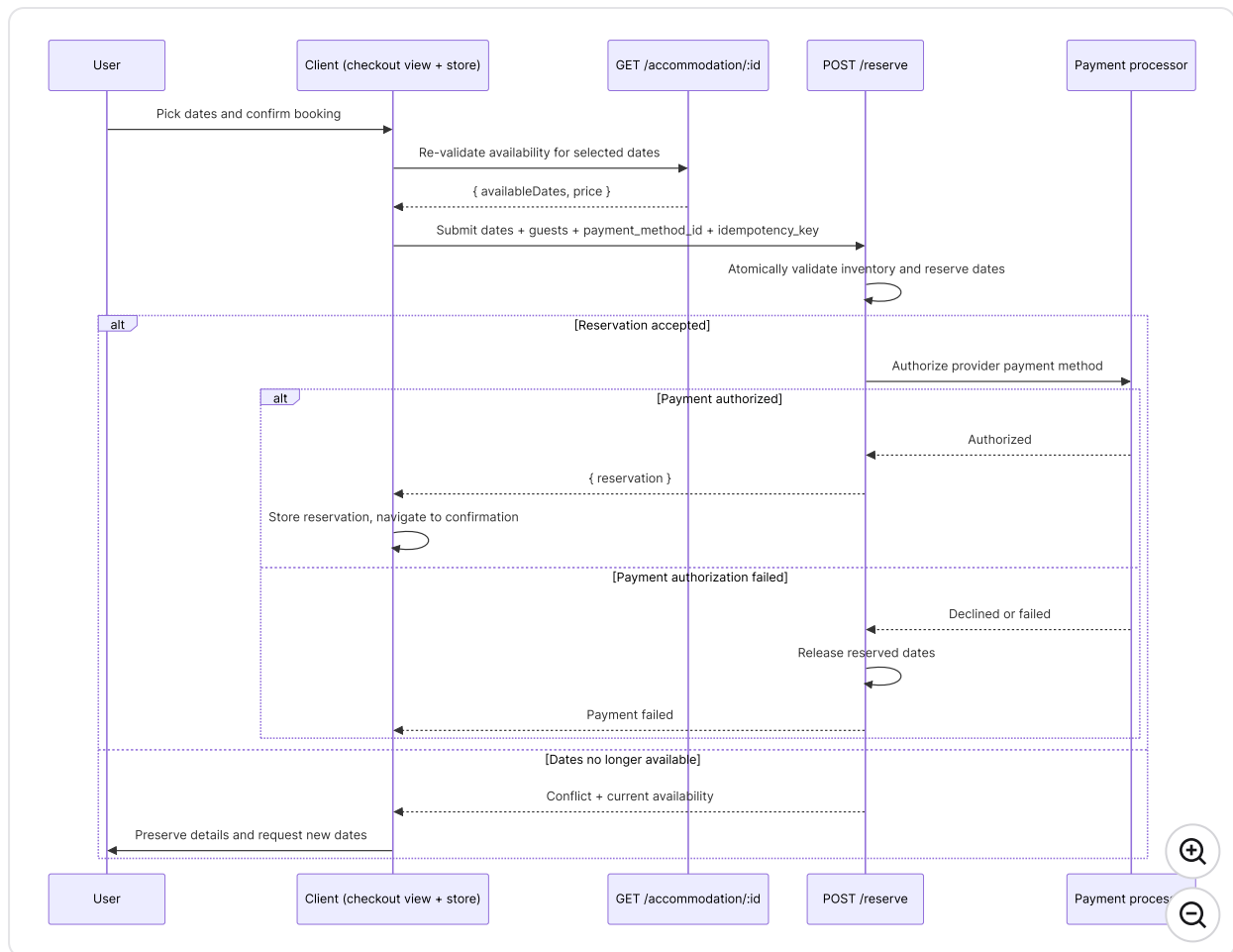
Parameter	Type	Description
dates	mixed*	Dates to reserve the accommodation
guests	number	Number of guests covered by the reservation
payment_method_id	string	Opaque identifier created by provider-hosted payment fields
idempotency_key	string	Stable key reused for retries of this reservation attempt

* The format of the `dates` field should be consistent with the selected date range format in the search API.

The reservation flow spans availability, payment, and confirmation. For this interview scope, the displayed price remains unchanged during the booking attempt. The earlier availability `GET` is a UX hint; correctness comes from `POST /reserve` atomically validating the date range, reserving the dates, and deduplicating retries by idempotency key.

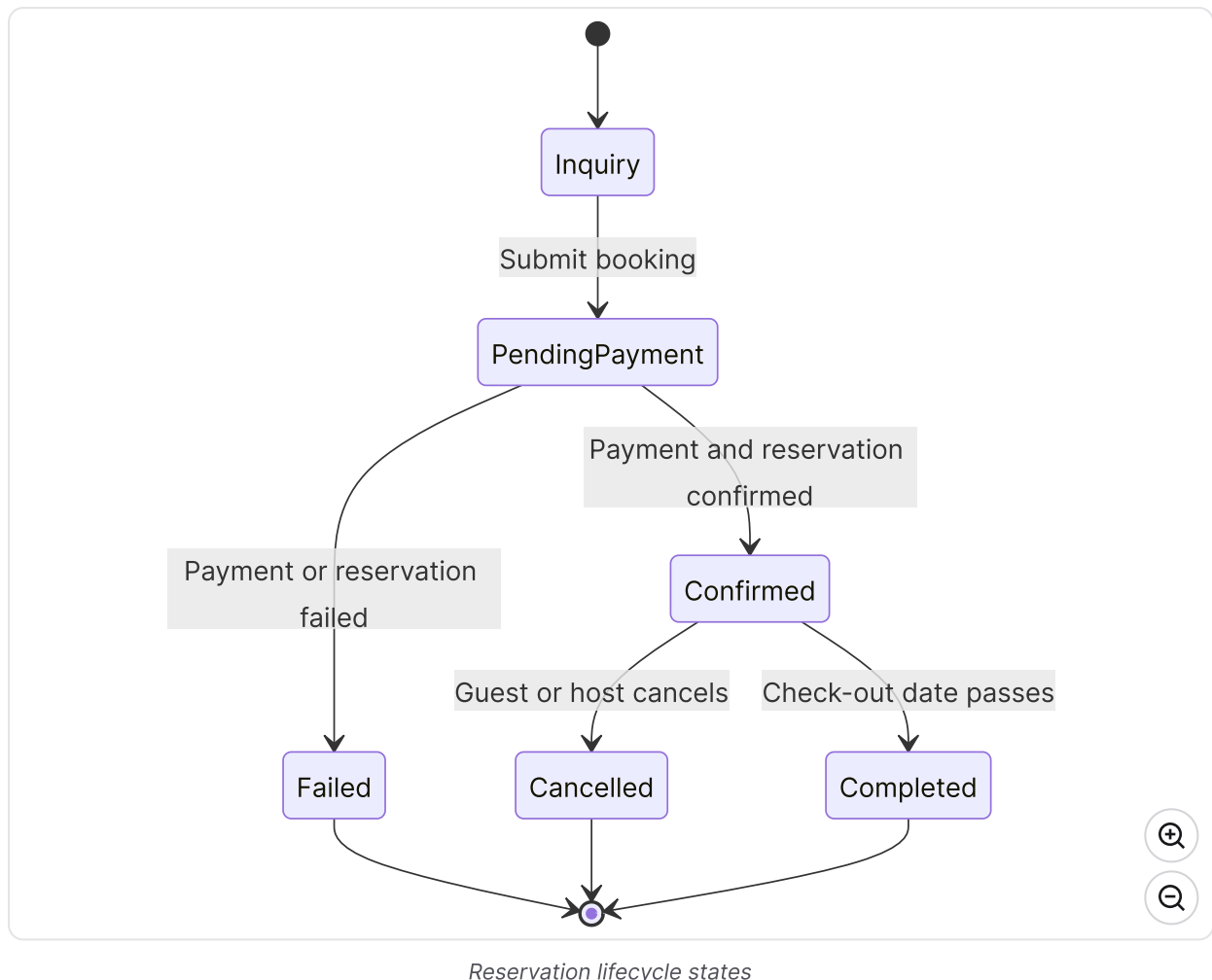
If the dates are no longer available, the endpoint returns a conflict with current availability. The client preserves the other booking details and asks the user to choose available dates before retrying.

If payment orchestration becomes the focus, extend this flow with hold expiry, authenticated provider webhooks, and compensation when payment succeeds but booking confirmation fails.



Reservation flow with availability re-check and payment

A reservation moves through a small number of states between inquiry and completion, and the client UI (confirmation page, trip dashboard, cancellation controls) is driven by this status field.



Sample response

A reservation object is returned upon successful reservation.

```
1  {
2    "id": 456, // Reservation ID.
3    "total_price": 400,
4    "currency": "USD",
5    "dates": {
6      "check_in": "2022-12-24",
7      "check_out": "2022-12-27",
8    },
9    "accommodation": {
10     "id": 561602,
11     "address": {
12       "country": "US",
13       "street_address": "888 Brannan Street",
14       "city": "San Francisco",
15       "zip": "94103",
16       "state": "CA",
17       // ... Other address fields.
```

```
18     },
19   },
20   "payment_method": {
21     "brand": "visa",
22     "card_last_four_digits": "1234",
23   },
24 }
```

Exercise 1 of 6 Beta

A travel search accepts either free-text destinations or map boundary coordinates and uses an external geocoding provider for text. Where should the provider call happen?

- 1 Call the provider directly from the browser with a referrer-restricted API key, then send coordinates to the search service.
- 2 Geocode in the browser and have the application server validate only the returned coordinates before searching.
- 3 Send the location to the application server, which calls the provider with a private key behind caching and rate limits.

[Reveal answer](#)[Check answer ↩](#)

Optimizations and deep dive

To recap, the most important aspects of a travel booking website are search discovery, global usage, performance, device support, booking UX, accessibility, and forms:

- **Search engine optimization (SEO)**: This section explains how search and listing pages stay crawlable, shareable, and useful for organic discovery.
- **Internationalization (i18n)**: This section covers locale, language, currency, dates, addresses, and region-specific travel expectations.
- **Performance**: This section discusses loading speed, image delivery, Core Web Vitals, and performance tradeoffs that affect conversion.

- **Device support**: This section covers responsive images, mobile layouts, and cross-device booking behavior.
- **User experience**: This section explains search context preservation, date-range selection, optimistic reservation flows, and high-friction travel UX details.
- **Accessibility**: This section covers image alternatives, forms, controls, and keyboard access across search and booking flows.
- **Form optimizations**: This section discusses checkout-style form improvements for guest details, payment, validation, and autofill.
- **Other topics**: This section lists adjacent travel-product concerns that are useful follow-up discussion points.

Search engine optimization (SEO)

Travel sites rely heavily on organic traffic, so search result pages and listing pages both need to be crawlable and shareable, starting with making search URLs bookmarkable.

Bookmarkable search results

When searching on these travel sites, notice that the search query and filters are reflected in the URL. If you load the URL in a new window, the same results and search query are shown. Syncing the search query with the URL is an important feature because:

1. **Bookmarking**: Allows the particular search to be bookmarked, which is a common action users take when doing research for travel.
2. **Deep linking**: Other sites like travel blogs can link to the results page of specific search filters (e.g. vacation rentals in San Francisco). This helps improve the site's SEO and discoverability.

★ Treat the URL as the source of truth for search filters

Location, dates, guests, and amenity filters all belong in the query string or path so the same state can be bookmarked, shared, and restored on reload. Only selected durable destination pages should be indexed. Keeping filters only in client memory breaks back/forward navigation and forces users to re-enter their criteria whenever they open a listing in a new tab.

Pre-generate pages for popular searches

When you search for "Vacation in San Francisco" in a search engine, you will most likely see results from popular travel sites like Expedia, Airbnb, and TripAdvisor, among many others. These pages aren't content articles; they're the site's search results page with some pre-filled filters. How is this accomplished?

To improve SEO by appearing relevant to what users are searching for, travel sites generate dedicated pages for popular destination searches that show relevant listings for these terms. For example, Airbnb has generated dedicated pages for "Vacation rental apartments in San Francisco" and "Vacation rental apartments in New York". Selected destination pages can be included in a sitemap so search engines can discover them. Transient dates, guests, sort order, map bounds, and low-value filter combinations should remain shareable without being indexed.

URLs are commonly implemented in one of two ways:

- **Readable URLs:** <https://www.airbnb.com/san-francisco-ca/stays>
- **"Ugly" URLs:** <https://www.airbnb.com/search?location=123> (where 123 is an internal ID mapping to San Francisco city)

Airbnb uses readable URLs because they are easier for users and crawlers to understand. However, putting a keyword in the path does not by itself guarantee a better ranking.

✓ **Prefer readable URLs for popular search landing pages**

A path like `/san-francisco-ca/stays` is easier to understand than `/search?location=123`. Reserve opaque query-parameter URLs for internal or transient routes that should not be indexed.

Dynamic rendering as a legacy workaround

Dynamic rendering sends crawlers a rendered page while serving users a different client-rendered page. Google no longer recommends it as a long-term solution because maintaining two rendering paths creates parity and operational risks. Prefer server-side rendering, static generation, or hydration so crawlers and users receive equivalent content.

For a legacy route that cannot be migrated immediately, dynamic rendering can be a short-term fallback with automated parity checks and a removal plan. The historical

Expedia Group Vrbo case study illustrates the technique: crawlers received a full page while users received above-the-fold content first and loaded the rest asynchronously.

Exercise Beta

Which choices support travel-site SEO without indexing every transient search state?
Select all that apply.

- 1

Return meaningful destination and accommodation content in initial HTML.

☐
- 2

Use clickable result-card containers for client navigation, keeping the destination URL only in application state.

☐
- 3

Provide stable canonical URLs and metadata for indexable pages.

☐
- 4

Give each live date and guest search its own self-referencing canonical URL so specific availability can rank independently.

☐
- 5

Prevent low-value or duplicate filter permutations from creating uncontrolled crawl space.

☐

[Reveal answer](#)[Check answer ↩](#)

Internationalization (i18n)

Some of the following is extracted from our [Quiz Questions](#) on [internationalization](#):

- Have dedicated pages translated in the supported languages.
- Make the language and country selector very prominent (e.g. in the navbar).
- For listing details that are contributed by users, add an automatic translation feature so that users visiting country X who don't speak country X's language can understand the listing in their native language.
- Set the `lang` attribute on the `html` tag (e.g. `<html lang="zh-cn">`) to tell browsers and search engines the language of the page, which helps browsers offer a translation of the page. This is also important for SEO.
- Enable locale-specific UI:

- Using the `:lang()` CSS pseudo-class to change display
- Right-to-left languages
 - Using CSS Logical Properties
 - HTML's `dir` attribute
 - CSS `direction: rtl`
- Consider differences in the length of text in different languages.
- Do not concatenate translated strings.
- Do not put text in images.
- Be mindful of how colors are perceived in different cultures.
- Treat check-in and check-out as property-local calendar dates. Use the property's IANA time zone for time-bearing rules rather than silently converting dates to the browser's time zone.
- Treat locale as a presentation preference. The server returns the amount and ISO currency code used for display and reservation.

Exercise Beta

Which concerns must a global travel-booking UI handle? Select all that apply.

- 1

Use logical layout and flexible controls for RTL and longer translations.

☐
- 2

Display server-provided prices, fees, and counts with locale-aware formatters.

☐
- 3

Interpret check-in rules and calendar dates in the property's intended time zone.

☐
- 4

Convert check-in and check-out midnights to the browser's zone before displaying or submitting them.

☐
- 5

Translate guest and night nouns separately, then concatenate them with locale-formatted counts.

☐



Reveal answer

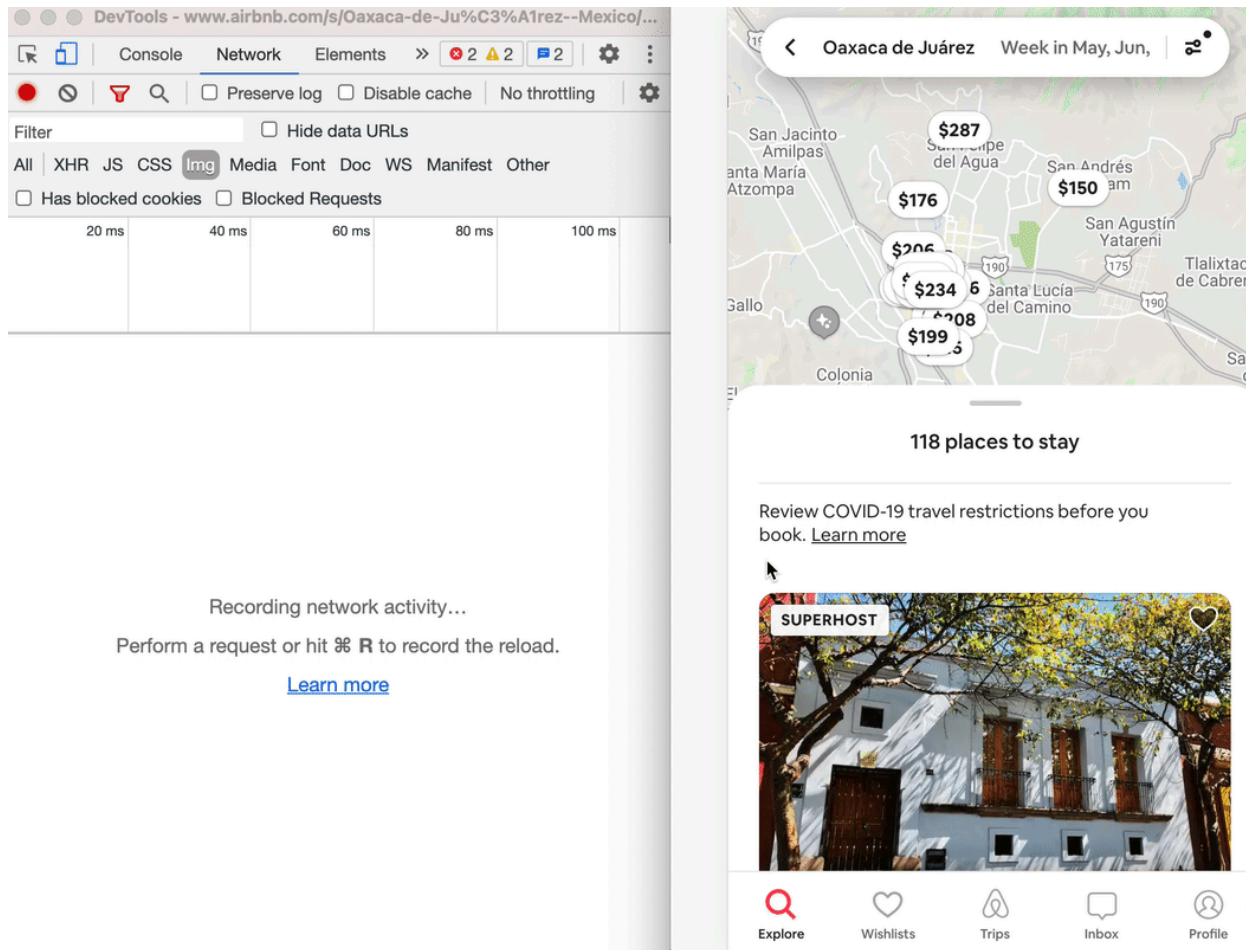
Check answer ←

Performance

Performance is known to affect conversions, so for websites where the aim is to get customers to make bookings, performance is important. Performance optimizations have been covered in great detail in earlier questions, so in this question we will list the relevant performance optimizations along with blog articles by companies should you want more details.

Image optimizations

- **Image carousel:** Photos are widely used on travel websites to showcase how attractive the destination/accommodation is. We have covered the implementation of image carousels in the [Image Carousel system design article](#).
- **Image preloading/lazy loading:** One super useful technique is to use JavaScript to have fine-grained control over when images load.
 - [Airbnb optimized the image carousel experience in their room listings with a combination of lazy loading and preloading:](#)
 1. Initially, only the first image is loaded (the remaining images will be lazily loaded).
 2. The second image is preloaded when the user shows possible intent of viewing more images:
 - Cursor hovers over the image carousel.
 - Focuses on the "Next" button via tabbing.
 - Image carousel comes into view (on mobile devices).
 3. If the user does view the second image (which signals high intent to browse even more images), the next three images (3rd to 5th) are preloaded.
 4. As the user clicks "Next" again to browse more images, the $(n + 3)^{\text{th}}$ image is preloaded.



Airbnb image carousel lazy loading example on mobile

- **Responsive images:** Serving the most suitable image for the device.
 - [Images on the Web: Part 1 — Responsive Images | Expedia Group Technology](#)
 - [Images on the Web: Part 2 — Implementing responsive images | Expedia Group Technology](#)
- **Image formats:** Use `webp` format for photos and SVGs for icons where possible.
 - [Choose the right image format | web.dev](#)

Code splitting

- [Expedia's Vrbo prioritizes above-the-fold content and loads code for it first.](#)
- Page/route-level code splitting to prevent huge JavaScript bundles if using a single-page app architecture.
- Lazy load UI components that are not shown on initial render: (1) below-the-fold elements (e.g. footer), and (2) elements that only appear after interaction (e.g. popover, modal).

Performance monitoring

- Use tools like Lighthouse and Web Vitals to profile websites and measure performance.
- Airbnb came up with their own Page Performance Score and defined the metrics that mattered for web performance.
- Set up performance budgets that run on CI.
- If caching becomes the focus, distinguish immutable media and public listing metadata from query-keyed search results, short-lived availability checks, and private reservations; each needs a different cache key and freshness policy.

React-specific tips

- React-specific performance optimizations done by Airbnb: React Performance Fixes on Airbnb Listing Pages | The Airbnb Tech Blog.
- 12 Tips to Improve Client Side Page Performance | Expedia Group Technology

Bundling optimizations

- Module federation: Using Webpack Module Federation to Create an App Shell | Expedia Group Technology
- Optimizing a Page: Resource Hints, Critical CSS, and Webpack | Expedia Group Technology

Virtual list/windowing for long lists with infinite scrolling

Use virtual list/windowing for long lists with infinite scrolling. Read more about List Virtualization on patterns.dev.

Progressive web apps

- Progressive Web Apps with Service Workers | Booking.com Engineering

Other performance tips

- When using client-side rendering, split up the listing response into multiple payloads so as to show results faster. To quickly show results, the response payload can be split into a few chunks, e.g. returning the first 5 results (or however many are above the fold), then loading the rest of the results on the page after that.

- Web Applications: Analyzing Client-Side Performance | Expedia Group Technology
- Go Fast or Go Home: The Process of Optimizing for Client Performance

Further reading

- What kind of things must you be wary of when designing or developing for multilingual sites?
- How do you serve a page with content in multiple languages?
- Building Airbnb's Internationalization Platform
- Adding support for Arabic and Hebrew languages on Airbnb

Exercise Beta

Which optimizations protect the search-to-booking path? Select all that apply.

- 1 Hydrate reviews, recommendations, experiments, and marketing widgets before enabling search results so the route becomes interactive consistently. ☐
- 2 Load the complete result set for the current filters once, then filter its map markers locally as the viewport moves. ☐
- 3 Show route-critical search or listing content before secondary recommendations and reviews. ☐
- 4 Reserve and serve appropriately sized listing and gallery images. ☐
- 5 Split code by route and defer below-the-fold or interaction-only components. ☐
- 6 Fetch and render results for the active page or map viewport, virtualizing long lists where needed. ☐



Reveal answer

Check answer ↩

Device support

- **Responsive images:** Serving the most suitable image for the device, as mentioned in the image optimization section above.
 - [Images on the Web: Part 1 — Responsive Images | Expedia Group Technology](#)
 - [Images on the Web: Part 2 — Implementing responsive images | Expedia Group Technology](#)
- **Device-specific UI:** Showing different UI for different devices, which can involve using a totally different information architecture.
 - Dynamic number of result items in a row and per page depending on device width and height.
 - Since maps are hard to use on smaller devices due to the limited screen real estate, consider not having a map UI at all. Without the map, clients can avoid loading map code and textures entirely on mobile devices.
 - Airbnb's listings are more app-like on mobile, with a floating bottom bar. The listing page also does not open listings on a new page, presumably because it is harder to manage many open tabs on mobile.
 - Support swiping on image carousels on mobile.
 - A less aggressive preloading strategy on mobile since mobile data can be more expensive.
 - Interactive elements should be larger on mobile.

Exercise Beta

Which responsive strategy best supports booking on both mobile and desktop?

- 1 Maintain a separate mobile booking flow with simplified search parameters and its own reservation payload.
- 2 Reuse the search and booking state model while adapting filter presentation, touch targets, galleries, and sticky actions by viewport.
- 3 Preserve the desktop layout and scale it down on phones so every feature remains in the same visual position.

[Reveal answer](#)[Check answer ↩](#)

User experience

A few travel-specific UX details matter disproportionately: preserving the user's search context when they dive into a listing, date-range pickers, and optimistic reservation flows.

Preserving search context

Travel shoppers tend to compare multiple listings after narrowing the results. Opening details without losing filters, map position, and the current result location lets them return to that comparison quickly.

Use an ordinary link to listing details and preserve URL-backed search state plus an item anchor for restoration. Users can choose a new tab with standard browser controls.

The alternative is to open the listing details within a full-screen modal on the same page and shallowly update the URL (via `History.pushState()`), similar to [Zillow.com](https://www.zillow.com) and [Instagram.com](https://www.instagram.com). Dismissing the modal will reveal the search results behind it, and users can continue browsing the results. The downside of this approach is that it is more complicated to implement and will require more JavaScript on the client since everything is on one single page and the listing details need to be fetched via AJAX.

✓ Preserve search context without overriding browser navigation

Keep filters, continuation, map viewport, and the invoking listing anchor restorable. A shallow modal is an optional product choice, not the only way to preserve context.

Exercise Beta

A user opens a listing from filtered search results and then returns. What should the app preserve?

- 1 Start a fresh search on return so availability and ranking are current, resetting the user to the first result.
- 2 Keep only the old scroll offset and reapply it after fetching the default result list.

3

Restore destination, dates, guests, filters, result continuation, and a sensible scroll position.



Reveal answer

Check answer ↩

Accessibility

- Accommodation photos are usually decision-relevant. Provide or facilitate concise descriptions, and use `alt=""` only for images that are truly decorative or redundant.
- Give destination, date, guest, and filter controls programmatic labels and instructions. Use appropriate input types, names, and `autocomplete` values in reservation forms.
- Make the date picker fully keyboard-operable and expose selected and unavailable dates without relying only on color.
- Keep a list alternative to the map and announce meaningful result, availability, and reservation-status changes without moving focus unexpectedly.
- Associate validation errors with their fields, move focus to a useful recovery point, and preserve safe form progress across steps and transient failures.
- The [W3C Web Accessibility Initiative tutorials](#) cover page structure, menus, images, tables, forms, and carousels with current standards-based guidance.

If accessibility becomes a deeper discussion, expand the date-picker keyboard model and the synchronization between the map and its equivalent results list.

Exercise Beta

Which practices make a travel search and reservation flow accessible? Select all that apply.

1

Implement complete keyboard behavior and clear selected and unavailable date states.

☐

2

Announce meaningful search-result and booking-status changes without moving focus unexpectedly.

☐

3

Make the map the primary results control and expose listing details only after a marker is selected.

☐

4

Give destination, date, guest, and filter controls programmatic labels and instructions.

☐

5

Use muted and highlighted colors for unavailable and selected dates while leaving every calendar cell otherwise identical.

☐

Reveal answer

Check answer ↩

Form optimizations

Form optimizations have been covered in detail in [E-commerce website system design](#).

To recap:

- Country-specific address/payment forms.
- Optimize the autofill experience.
- Show error messages.
- Clear focus states.
- Read more about building good [Payment forms](#) and [general form best practices](#) on web.dev.

Exercise Beta

Which form behaviors reduce booking abandonment while preserving correctness?
Select all that apply.

1

Reset the reservation form after any availability refresh so no field is based on an outdated quote.

☐

2

Delay validation until final payment submission so earlier booking steps remain visually quiet.

☐

3

Use input types, names, and autocomplete tokens suited to traveler and payment information.

☐

4

Place clear errors near controls and connect them programmatically.

☐

5

Retain recoverable form state across steps and transient submission failures.

☐



Reveal answer

Check answer ←

Other topics

- Map
 - Markers that are in close proximity can be clustered together.
- Whitelabelling
 - Building and scaling different travel websites with one codebase | Agoda Engineering & Design
 - Managing and scaling different white label development and testing environments | Agoda Engineering & Design

Summary

A travel booking product like Airbnb has to be evaluated under two lenses that pull in different directions: an SEO lens that governs the crawlable, shareable public surfaces, and a conversion lens that governs the signed-in, transactional flow from search through reservation. Travel is unusual in that the same product has to win organic traffic on long-tail city and neighborhood queries and then move a logged-in guest through dates, guests, price, and payment without losing them. A design that optimizes only one lens either leaks acquisition or loses revenue at the checkout step, so the interview answer has to hold both simultaneously.

Through the SEO lens. Public destination and listing detail pages default to server-side rendering with hydration so crawlers see complete HTML, the first contentful paint arrives before hydration, and subsequent interactions run on the client. Popular destinations are served from pre-generated landing pages under readable URLs like `/san-francisco-ca/stays` and surfaced to search engines through `sitemap.xml`. The URL is treated as the source of truth for search filters, including location,

`check_in` / `check_out` dates, guest count, and amenities, so filtered results are bookmarkable and shareable. Only durable destination and listing pages with unique value are made indexable. Canonical tags, locale-aware `hreflang`, and `lang / dir` attributes round out the discoverability story across translated markets.

Through the conversion lens. The `/search` endpoint accepts either free-text location resolved through a server-side geocoding proxy or explicit bounding-box coordinates, along with separate `check_in` and `check_out` parameters so date validation and property-time-zone handling stay explicit. The client-side data model keeps `Listing` referencing `Host` and `Location`, binding to an `AvailabilityCalendar`, and participating in `Booking` records so the reservation lifecycle can be reasoned about without refetching. `POST /reserve` atomically validates and reserves the selected dates before payment authorization; if authorization fails, it releases those dates. Airbnb-style image carousels load the first image eagerly, preload the next on hover or focus intent, and lazily request the remainder, keeping the gallery feeling instant without blowing the image budget. Map and list stay synchronized through a shared client store of results, with marker clustering at low zoom levels and viewport-driven `/search` calls as the user pans. The server-provided currency is formatted for the user's locale, and device-specific UI can remove the map on small screens, use a floating bottom action bar, and enlarge touch targets for thumb reach.

Where the two lenses agree is the URL-backed search state used by `/search` and preserved while navigating to `/accommodation/:id`. It lets guests share, bookmark, reload, and return to their results without losing context, while `/reserve` remains a separate mutation contract rather than URL state. Rendering, API shape, and data model line up behind those boundaries.

References

The references below group sources by topic, so it is easier to map each source back to the part of the article it informed.

Airbnb frontend architecture and performance

These references inform the rendering strategy, code-splitting, and client performance choices discussed throughout the architecture and optimizations sections.

- [Researchitecting Airbnb's frontend](#) for the overall frontend platform direction behind airbnb.com and the tradeoffs that shaped its client architecture.
- [Building a faster web experience with the postTask scheduler](#) for scheduling long-running work off the main thread on interactive booking pages.
- [Server rendering, code splitting, and lazy loading with React Router v4](#) for the SSR and route-based code-splitting approach relevant to listing and search pages.
- [Isomorphic JavaScript: the future of web apps](#) for the isomorphic rendering background used in the rendering approach discussion.
- [Creating Airbnb's page performance score](#) for how a large travel site turns performance into a shared internal metric.
- [Measuring web performance at Airbnb](#) for the RUM and lab measurement practices behind the performance work referenced in the article.
- [React performance fixes on Airbnb listing pages](#) for concrete listing-page performance wins that map directly onto the listing detail design here.

Internationalization and localization

These references inform the i18n, currency, and locale considerations called out for a global travel product.

- [Building Airbnb's internationalization platform](#) for the platform-level approach to translations, message management, and locale routing.
- [Adding support for Arabic and Hebrew languages on Airbnb](#) for the RTL layout, typography, and bidirectional text considerations relevant to global travel sites.

Expedia and other travel-site case studies

These references provide cross-company context for images, bundling, and resource prioritization on travel sites with a similar listing-heavy shape.

- [Images on the web: Part 1 — responsive images](#) for the responsive image fundamentals behind listing media.
- [Images on the web: Part 2 — implementing responsive images](#) for the implementation details of `srcset`, `sizes`, and art direction on large travel pages.
- [Expedia's Vrbo prioritizes above-the-fold contents](#) for concrete above-the-fold prioritization techniques on a travel homepage.

- [12 tips to improve client side page performance](#) for a practical checklist of client performance fixes applicable to listing and search pages.
- [Using Webpack module federation to create an app shell](#) for a micro-frontend style app-shell architecture on a multi-team travel codebase.
- [Optimizing a page: resource hints, critical CSS, and Webpack](#) for resource hints and critical CSS techniques referenced in the loading discussion.
- [Progressive web apps with service workers | Booking.com](#) for a booking-focused view on service workers, offline behavior, and caching.
- [Breaking the monolith — modular redesign of Agoda.com](#) for how another travel site evolved from a monolithic frontend into modular surfaces.
- [Yes I'm lazy | TripAdvisor Engineering](#) for lazy-loading patterns on image-heavy travel listing pages.

Form best practices for booking flows

These references back the payment, address, and autofill guidance in the form optimizations section.

- [Payment and address form best practices | web.dev](#) for end-to-end guidance on payment and address fields in a checkout-style flow.
- [Address forms | web.dev](#) for the address-specific field structure and locale-aware layout recommendations.
- [Autofill on browsers: a deep dive | eBay](#) for the browser autofill behavior and attribute choices that directly affect booking form completion rates.

Related system design articles

The articles below dive deeper into mechanics that are touched on but not fully explored here.

- [E-commerce \(Amazon\)](#) for the broader checkout, form, and catalog patterns that a booking flow reuses and adapts.
- [Photo Sharing \(Instagram\)](#) for the image loading, lazy loading, and media-heavy feed techniques that apply to listing galleries.

Exercises

Exercise 1 of 18 Beta

A traveler-facing booking design focuses on the journey from search through reservation. Which boundaries keep the work focused? Select all that apply.

1

Include host listing management, payout reporting, and property onboarding in the first-version traveler experience.

☐

2

Cover search and filtering, listing details and availability, reservation, and the resulting confirmation.

☐

3

Require live bidding and repeated price negotiation while a traveler completes the reservation form.

☐

4

Hold the displayed stay price fixed during one reservation attempt.

☐

5

Treat SEO, internationalization, and route-specific performance and reliability as first-version constraints for public search and listing pages.

☐

6

Allow anonymous public discovery across devices while requiring sign-in when the traveler books.

☐

Reveal answer

Check answer ↩

Discussions

↩ ↪ | Normal T_T | B I U | 🔗 | ✎ <> Aa ☰ ☷ </> ☰

Type comment here...

0/1000

Post



No comments yet

Start the discussion with a question or helpful note.

 Report an issue



✓ Mark complete



Subscribe to our newsletter

Enter your email

Notify me

Practice

Get started

JavaScript functions

User interface coding

System design

Quiz

Guides

Front End Interview Playbook

Front End System Design Playbook

React Interview Playbook

Behavioral Interview Playbook

Study Plans

1 Week Plan

1 Month Plan

3 Months Plan

Company

Pricing

Promotions

Roadmap

About

Team

Contact us

Become an affiliate

Careers

Blog

Medium

DEV Community



English (US) ▾